

Arquitectura de Microservicios con Contenedores

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Comunicación Asincrónica y Kafka

Temario

1. ¿Por qué comunicación asíncrona?
2. Fundamentos de mensajería
3. Event-Driven Architecture (y CQRS light)
4. Saga en la práctica: coreografía vs. orquestación
5. Cómo funciona Kafka
6. Patrones de confiabilidad (Outbox, Inbox, contratos, lag)
7. Cierre, checklist y anti-patterns

**¿Por qué comunicación
asíncrona?**

¿De dónde venimos?

En la clase 6 medimos qué pasa cuando llegan más requests de los que se pueden procesar.

- **Sync bajo sobrecarga:** perdió 95 de 100 requests.
- **Con una cola en el medio:** perdió 0, pero la latencia se fue a 30 s.

En la clase 7 vieron un flujo de venta coreografiado – reaccionando a eventos, sin broker de por medio todavía.

Hoy: cerramos el círculo. Qué hay realmente detrás de esos eventos, y qué se rompe si no lo diseñás con cuidado.

Sync vs Async – comparación

Aspecto	Síncrono	Asíncrono
Acoplamiento temporal	Alto: emisor y receptor deben estar up a la vez	Bajo: el broker media en el tiempo
Latencia percibida	La del eslabón más lento del camino	La del primer hop (encolar)
Falla en cascada	Se propaga directo	Amortiguada por el broker
Complejidad operativa	Baja	Alta: broker, DLQ, idempotencia, observabilidad
Trazabilidad del flujo	Directa (call stack)	Indirecta (hay que reconstruir la cadena de eventos)

Comunicación Asíncrona

- El emisor **no espera** respuesta inmediata.
- Brinda desacoplamiento **temporal** y **espacial**.
- Necesita un intermediario: **Message Broker**.
- Mayor resiliencia, menor acoplamiento.
- Mayor complejidad operacional.

Message Brokers conocidos

- **RabbitMQ:** broker tradicional, flexible.
- **Apache Kafka:** log distribuido, alto throughput.
- **NATS:** ultraliviano y cloud-native.

¿Cuándo NO usar async?

- Necesitás la respuesta **ya**, para decidir el siguiente paso.
- El flujo es simple y un único consumidor alcanza.
- No podés pagar el costo operativo de un broker (equipo, monitoreo, on-call).
- La consistencia inmediata importa más que el desacople.

Async no es progreso automático – es un trade-off. Se paga con complejidad operativa lo que se gana en resiliencia y escala.

Fundamentos de mensajería

Queue vs Pub/Sub – dos modelos distintos

Característica	Cola (Queue)	Publish / Subscribe
Entrega	1 mensaje → 1 grupo consumidor	1 mensaje → N suscriptores (muchos grupos)
Caso de uso	Distribuir trabajo entre workers	Notificar a múltiples servicios
Ejemplo	Procesar un pago (un worker)	OrderCreated → pago, email, analytics
Herramienta	RabbitMQ queues, AWS SQS	Kafka topics, RabbitMQ exchanges

Anatomía de un mensaje: CloudEvents

Especificación de la **CNCF** para describir eventos.

- Atributos comunes: id, source, type, specversion, data.
- Agnóstico de transporte y de lenguaje.
- Usable sobre HTTP, Kafka, AMQP, NATS.

Convención de cátedra: todos los labs event-driven usan CloudEvents, y los que tienen comunicación asíncrona incluyen un `asynapi.yaml`.

```
order-created.json

{
  "specversion": "1.0",
  "id": "3f8a1e02-19b0",
  "source": "/orders-service",
  "type": "com.itba.order.created",
  "time": "2026-09-25T14:03:00Z",
  "datacontenttype": "application/json",
  "data": {
    "orderId": "1234",
    "customerId": "cust-88",
    "total": 15990.0
  }
}
```

NATS: messaging ultraliviano

Problema: Kafka y RabbitMQ son potentes pero pesados para casos simples.

- Un binario Go, sin dependencias, arranca en ~2 s.
- Pub/Sub + Request-Reply + Queue Groups.
- **JetStream:** persistencia opcional. Sin él, el mensaje se pierde si no hay nadie consumiendo.
- Docker Compose de 4 líneas.

NATS core (sin JetStream)

Publisher → NATS → Subscriber

Nadie escuchando = mensaje perdido.

NATS JetStream

Publisher → Stream (persistente) → Consumer (offset)

Sobrevive a un consumidor caído.

NATS vs RabbitMQ vs Kafka

Aspecto	NATS	RabbitMQ	Kafka
Modelo	Pub/Sub + Request-Reply	Broker con exchanges	Log distribuido
Persistencia	Opcional (JetStream)	Sí (hasta consumir)	Sí (retención configurable)
Setup	1 binario, 0 config	Erlang, plugins	Cluster de brokers
Ideal para	Messaging simple y rápido	Routing flexible	Alto throughput, replay, event sourcing

Event-Driven Architecture

Event-Driven Architecture (EDA)

- **Sistemas reactivos:** los servicios emiten *eventos* describiendo hechos ocurridos. Otros servicios reaccionan.
- **Desacoplamiento:** el emisor no sabe, ni le interesa, quién está escuchando.

Distinción importante:

- **Eventos:** "algo pasó" (OrderCreated) → pasado, inmutable, sin expectativa.
- **Comandos:** "hacé algo" (ProcessPayment) → imperativo, espera resultado.

La coreografía que ya vieron, es EDA

En la clase 7, se mostró este flujo – sin broker, sin pub/sub, pero con la noción de evento ya instalada:



Eso es EDA aplicada a un flujo de negocio: cada servicio reacciona a lo que pasó, nadie orquesta desde afuera.

Lo que faltaba en esa clase: un broker real transportando esos eventos. Eso viene ahora.

Tres formas de llevar información en un evento

- **Event notification:** "algo pasó", payload mínimo (`OrderCreated { orderId }`). El que escucha, si necesita más, pregunta.
- **Event-carried state transfer:** el evento lleva el estado completo (`OrderCreated { orderId, items, total, customer }`). Nadie necesita volver a preguntar.
- **Comando:** no es un evento – es una instrucción dirigida con expectativa de resultado (`ChargePayment`).

Trade-off: notification es liviano pero genera llamadas de vuelta; state transfer es autosuficiente pero duplica datos entre servicios.

Claim check: cuando el payload no cabe en el bus

State transfer sin límites termina en eventos gigantes (imágenes, PDFs, PII pesada).

- El evento lleva un **puntero** (`uri`, `blobId`) al payload completo en un store (S3, DB, object storage).
- Quien necesita el detalle lo descarga; el resto solo reacciona al hecho.
- El bus transporta el *hecho*, no el archivo.

Complementa notification vs state transfer: no es todo-o-nada – podés llevar metadata rica y dejar el blob afuera.

CQRS light: un write, muchos reads

- **Command Query Responsibility Segregation:** el modelo que escribe no tiene por qué ser el que se consulta.
- El servicio de escritura emite eventos; uno o más **read models** (projections) los consumen y materializan vistas optimizadas para lectura.
- Kafka encaja natural: el mismo log alimenta facturación, analytics y una proyección de "mis órdenes" sin acoplar al writer.

No es Event Sourcing completo (el historial como única fuente de verdad). Es separar write path de read path – y el bus es el puente.

Saga en la práctica

El flujo que ya conocen

Mismo dominio de la clase 7: venta → reserva de stock → cobro, con compensación si algo falla.

Estado	Evento disparador	Compensación si falla
SALE_REQUESTED	Cliente confirma la compra	–
Reserva de stock	STOCK_RESERVED	–
Cobro	PAYMENT_FAILED	ROLLBACK_INVENTORY
Cierre	–	CANCELED

Esto ya lo vieron como diagrama de estados. Ahora: ¿qué es esto en la práctica, cruzando la red?

Lo mismo, con mensajes reales

Cada flecha del diagrama de clase 7 es, en la práctica, un evento CloudEvents publicado en un topic de NATS.

- `orders.inventory.reserve-requested`
- `orders.inventory.reserved`
- `orders.payment.failed`

Nadie llama a nadie directamente – cada servicio se suscribe al topic que le importa.

```
stock-reserved.json

{
  "specversion": "1.0",
  "id": "8b21f0",
  "source": "/inventory-service",
  "type": "com.itba.inventory.reserved",
  "data": {
    "orderId": "1234",
    "sku": "FIG-045",
    "qty": 1
  }
}
```

Compensación como cadena de eventos

Coreografía pura: no hay nadie "arriba" decidiendo.
Cada servicio reacciona al evento de falla.

1. Payment publica PaymentFailed.
2. Inventory está suscripto a PaymentFailed → publica InventoryReleased.
3. Orders está suscripto a InventoryReleased → publica OrderCancelled.

Tres saltos, tres servicios, **ningún dueño único** del flujo de compensación.

```
payment-failed.json

{
  "specversion": "1.0",
  "id": "c4a190",
  "source": "/payment-service",
  "type": "com.itba.payment.failed",
  "data": {
    "orderId": "1234",
    "reason": "insufficient_funds"
  }
}
```

Un caso real: el pedido en Uber

- Uber construyó **Cadence** (~2016), un motor de orquestación de workflows durable, para flujos como el ciclo de vida de un pedido: reservar, cobrar, asignar repartidor, entregar.
- Los mismos autores de Cadence luego fundaron **Temporal**, su sucesor open source – hoy usado por Netflix, Snap, Stripe, Datadog.
- La razón de fondo: la coreografía pura, a medida que un flujo de negocio crece en pasos, se vuelve difícil de trazar y de debuggear. Exactamente lo que anotamos en la slide anterior.

Orquestación: el mismo flujo, otro código

Ya no hay eventos que alguien más escucha. Un **workflow** llama a **activities** como funciones normales – la orquestación es explícita, no implícita.

La compensación ya no es "publicar un evento y esperar que alguien reaccione": es una rama de control de flujo, en el mismo lugar donde está el resto de la lógica.

```
order-saga.workflow (pseudocódigo)

workflow OrderSaga(order):
    reserveInventory(order)

    try:
        chargePayment(order)
    except PaymentFailed:
        releaseInventory(order)    # compensación inline
        cancelOrder(order)
    return

    assignDelivery(order)
```

Durabilidad: lo que un motor de orquestación agrega

- El estado del workflow se **persiste en cada paso**, no solo vive en memoria.
- Si el worker que ejecuta OrderSaga se cae justo después de reserveInventory, al reiniciar retoma **exactamente ahí** – no se pierde la compensación, no se cobra dos veces.
- Esta durabilidad es la razón de ser de Cadence/Temporal: no es "un scheduler más", es lo que hace confiable ejecutar una Saga larga sin reinventar checkpoints a mano.

Conecta con la sección 6: esto es la misma preocupación que resuelve el *outbox pattern*, pero aplicada al orquestador en vez de a un único servicio.

Timeouts y deadlines en la Saga

Una Saga sin deadline puede quedar colgada para siempre: pago que no responde, repartidor que nunca acepta.

- Cada paso (y la Saga entera) tiene un **timeout** explícito.
- Al vencer: rama de compensación (liberar stock, cancelar orden) – igual que un `PaymentFailed`.
- En Temporal/Cadence: timers durables; sobreviven al restart del worker.

Sin timeout, "compensación" es teoría: en la práctica el flujo se pudre en un estado intermedio.

Coreografía vs Orquestación – la misma decisión, dos vistas

Aspecto	Coreografía (clase 7)	Orquestación (Temporal)
Dueño del flujo	Implícito, repartido entre servicios	Explícito: un workflow
Debugging	Difícil, sin owner único	Directo: un trace por Saga
Acoplamiento	Bajo – cada servicio conoce solo su parte	El orquestador conoce a todos los pasos
Escalabilidad / autonomía de equipos	Alta	Media – depende del orquestador
Conviene cuando...	Pocos pasos, equipos autónomos	Sagas largas, muchos pasos, necesitás visibilidad end-to-end

Cómo funciona Kafka

¿Por qué existe Kafka?

- Los sistemas modernos necesitan intercambiar datos continuamente.
- Las integraciones punto a punto generan alto acoplamiento.
- Productores y consumidores no siempre trabajan al mismo ritmo.
- Muchas aplicaciones necesitan reaccionar en tiempo real **y** conservar el historial para reprocesar.

Kafka aparece cuando integrar sistemas a gran escala empieza a ser un problema en sí mismo.

La idea central: el log

- Un log es una secuencia **ordenada** de registros **inmutables**.
- Se escribe solo al final (*append-only*).
- Responde dos preguntas: **¿qué pasó?** y **¿en qué orden?**
- Leer no elimina el dato – distintos consumidores pueden leer el mismo historial, cada uno a su propio ritmo.

Consumir no es sacar un mensaje: es avanzar una posición de lectura.

Topics y particiones

- Un **topic** agrupa eventos del mismo tipo (order-created, payment-approved).
- Un topic **no es una cola**: los eventos permanecen según la política de retención, y cada consumidor lleva su propio offset.
- Un topic se divide en **particiones** para paralelizar.
- Kafka garantiza orden **dentro de una partición** – no entre particiones de un mismo topic.

La key decide la partición

- $\text{partición} = \text{hash}(\text{key}) \% \text{cantidad_de_particiones}$
- Misma key → misma partición → **orden relativo preservado** para esa entidad.
- Ejemplos típicos de key: orderId, customerId, accountId.
- Sin key (o key distinta cada vez), Kafka reparte round-robin – sin garantía de orden entre esos eventos.

PRODUCER

Elegí una key y seguí sus eventos dentro de la partición.

Enviar order-101

Enviar order-202

Enviar order-303

Reset

bytes(key)

→

Murmur2

→

(hash & 0x7fffffff) % 3

→

partición

PARTICIÓN 0

0 records

vacía

PARTICIÓN 1

0 records

vacía

PARTICIÓN 2

0 records

vacía

Misma key → misma partición → **orden relativo preservado**. No existe orden global entre particiones.

Anti-pattern: key ausente o mal elegida

Después de la demo: el bug de ordenamiento casi nunca es "Kafka se rompió".

- **Sin key:** round-robin → eventos de la misma orden en particiones distintas → el consumer puede ver Paid antes que Created.
- **Key inestable** (`uuid()` por mensaje, o un campo que cambia): misma entidad salta de partición.
- **Key demasiado gruesa** (todo con `tenantId`): una partición caliente, el resto ocioso.

Regla: la key es parte del **contrato de orden**. Elegila por la entidad cuyo orden relativo importa (`orderId`, no "lo que haya a mano").

Brokers y replicación

- Un cluster Kafka está formado por varios **brokers**.
- Las particiones se distribuyen entre brokers – el topic es lógico, el broker es físico.
- Cada partición puede tener varias **réplicas**: una líder, el resto sigue.
- Si un broker cae, otra réplica asume como líder – así Kafka tolera fallos sin perder datos.

Broker 1 – P0 (líder), P1 (réplica)

Broker 2 – P1 (líder), P2 (réplica)

Broker 3 – P2 (líder), P0 (réplica)

Producers y Consumers

- El **producer** escribe records en un topic; con o sin key.
- El **consumer** lee con modelo *pull* – decide hasta dónde avanzó, y puede reiniciarse y continuar donde dejó.
- Los consumers se agrupan en **consumer groups** para repartirse el trabajo.
- Ninguno de los dos necesita conocer al otro – solo el topic.

Offsets: leer no es sacar

- Cada consumer lleva su propio **offset** (próxima posición a leer), guardado aparte del log.
- El log **no cambia** cuando se lee – dos consumers pueden estar en offsets distintos del mismo topic.
- Por eso Kafka permite **replay**: volver el offset a 0 sin perder ni duplicar el historial.

PARTICIÓN 0 · APPEND ONLY

Dos consumers leen el mismo historial a ritmos distintos.

[+ Producir evento](#)
[Reset](#)

LOG

0

OrderCreated

1

PaymentAp...

2

StockReserv...

3

OrderPacked

4

OrderShipped

5

DeliverySta...

next

pasado

append →

Facturación

próximo offset 4 · lag 2

[Leer siguiente](#)
[↶ Replay 0](#)


4

Analytics

próximo offset 2 · lag 4

[Leer siguiente](#)
[↶ Replay 0](#)


2

Leer o volver atrás mueve el cursor del consumer. **El log no cambia.**

Consumer Groups: paralelismo real

- Dentro de un mismo consumer group, **una partición se asigna a un solo consumer a la vez**.
- Paralelismo máximo = cantidad de particiones. Un quinto consumer con 4 particiones queda ocioso.
- Si un consumer cae, el grupo **rebalancea**: sus particiones se redistribuyen entre los que quedan.

CONSUMER GROUP · ORDERS

Dos consumers se reparten cuatro particiones. Seleccioná uno para simular su caída.

[+ Consumer](#)[Simular caída](#)[Reset](#)

TOPIC

P0

P1

P2

P3

↓ una partición, un consumer activo ↓



Consumer 1
activo

P0

P2



Consumer 2
activo

seleccionado

P1

P3

4 particiones

2 consumers activos

0 ociosos

0 rebalances

Paralelismo máximo = **cantidad de particiones**. Si un consumer cae, el grupo redistribuye su trabajo.

Competing consumers vs fan-out

Mismo consumer group = cola de trabajo (competing):

- Un mensaje lo procesa **un** miembro del grupo.
- Escala horizontal de workers sobre el mismo topic.
- Equivale al modelo "Queue" de la sección 2.

Distintos consumer groups = fan-out (pub/sub):

- Cada grupo recibe **todos** los mensajes.
- Facturación, email y analytics leen el mismo historial a su ritmo.
- Equivale al modelo "Pub/Sub" – y es lo que habilita CQRS.

Consumer lag: el termómetro del async

La demo de clase 6 ya lo midió: la cola no pierde requests, pero la latencia se fue a 30 s. Eso es **lag**.

- **Lag** = diferencia entre el final del log y el offset del consumer.
- Es la métrica operativa #1: no "¿el broker está up?", sino "¿estamos al día?".
- Respuestas típicas: más consumers (hasta #particiones), más particiones, optimizar el handler, o aceptar latencia bajo pico.

Kafka no es un buffer infinito e invisible: el lag es la factura del desacoplamiento temporal.

¿Cuándo usar Kafka?

Usar Kafka cuando...	No usar Kafka cuando...
Varios sistemas deben reaccionar al mismo evento	Una llamada síncrona simple alcanza
Necesitás desacople temporal y absorber picos	Necesitás request/response inmediato
Necesitás historial, auditoría o replay	No necesitás múltiples consumidores ni historial
El volumen de eventos es alto	El problema no justifica la complejidad operativa – NATS o una cola simple resuelven mejor

Patrones de confiabilidad

El problema: escribir en la DB y publicar el evento

Un servicio típico hace dos cosas al procesar una orden:

1. Actualiza su base de datos.
2. Publica un evento contándolo.

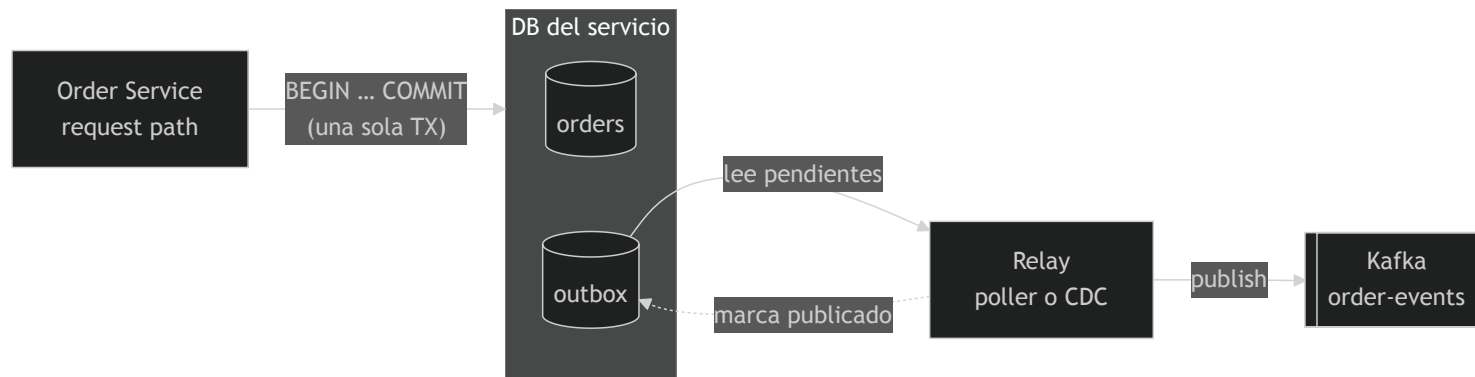
¿Qué pasa si el proceso se cae entre el paso 1 y el paso 2? La orden quedó guardada, pero nadie se enteró. O al revés: el evento salió, pero el commit falló.

Esto se conoce como el problema del **dual write** – dos sistemas distintos (DB y broker) que no comparten transacción.

Anti-pattern: publicar "a mano" después del commit (o antes) y esperar que no falle nunca. El patrón correcto es Outbox (o CDC).

Transactional Outbox

La idea: **no publicar el evento, guardarlo**. El evento pasa a ser una fila más en la misma DB que el cambio de negocio.



1. El request **escribe dos filas en una transacción**: la orden en orders y el evento OrderCreated en outbox. Commit → existen las dos. Rollback → ninguna.
2. Un proceso aparte (el **relay**) lee las filas pendientes de outbox y las publica al broker.
3. Recién cuando el broker confirma, el relay marca la fila como publicada.

La atomicidad se resuelve con la transacción de la DB. no con el broker.

Outbox en concreto

En el request path (código de la app):

```
BEGIN;
INSERT INTO orders (id, customer_id, total, status)
  VALUES ('o-42', 'c-7', 1500, 'CREATED');

INSERT INTO outbox (id, aggregate_id, type, payload)
  VALUES ('evt-9f1...',      -- CloudEvents id
          'o-42',           -- key de Kafka
          'order.created.v1',
          '{"orderId":"o-42","total":1500}');

COMMIT;
```

En el relay (loop aparte):

```
SELECT * FROM outbox
WHERE published_at IS NULL
ORDER BY created_at LIMIT 100
FOR UPDATE SKIP LOCKED;
-- publish a Kafka (key = aggregate_id), esperar ack
UPDATE outbox SET published_at = now() WHERE id IN (...);
```

¿Qué pasa si algo se cae?

Falla	Resultado
Crash antes del COMMIT	Ni orden ni evento.
Kafka caído	La orden entra; el evento espera en outbox.
Relay cae antes del publish	Sigue pendiente → se publica.
Relay cae entre publish y UPDATE	Se publica dos veces .

Garantía: **at-least-once** – nunca se pierde, pero puede duplicarse.
El consumidor deduplica por id (Inbox).

Dos formas de vaciar el Outbox

Poller

- Un worker hace SELECT periódico de filas pendientes y publica.
- Simple de implementar; latencia = intervalo de poll.
- Cuidado con locks y con marcar "publicado" de forma idempotente.

CDC (Change Data Capture)

- Herramientas como **Debezium** leen el write-ahead log de la DB y publican a Kafka.
- Sin código de aplicación publicando a mano.
- Menos latencia artificial; más pieza operativa (connector).

Transactional Inbox (lado consumidor)

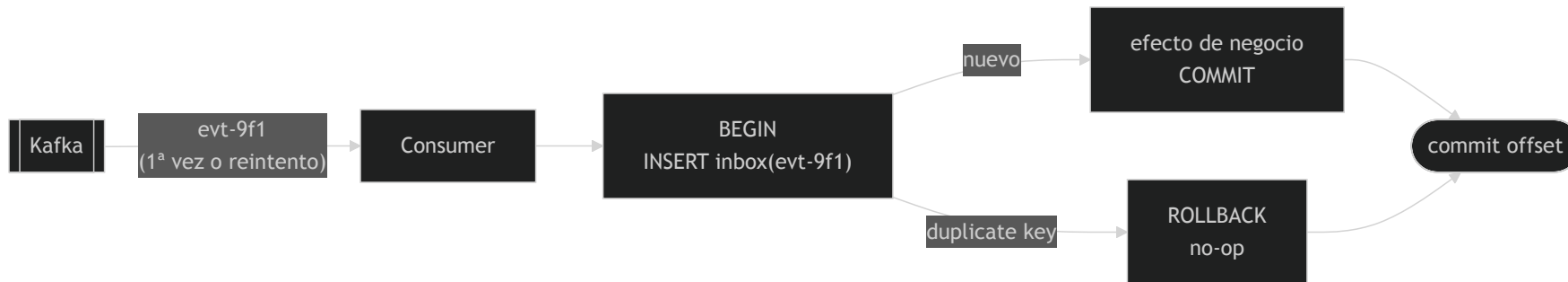
El espejo del Outbox, del lado que **consume**:

1. Llega el mensaje (puede ser un reintento).
2. En la misma transacción: INSERT en `inbox(message_id)` + efecto de negocio.
3. Si `message_id` ya existe → no-op (idempotente).

Ya lo vieron con Valkey en clase 7. Acá el contrato es el mismo, con DB.

Lado	Patrón	Evita
Productor	Outbox	"guardé y no avisé"
Consumidor	Inbox	"avisé y apliqué dos veces"

At-least-once en el bus + Outbox/Inbox en los bordes ≈ efecto exactamente una vez a nivel de negocio.



Consumidor idempotente: el contrato

Kafka (y casi todo broker) entrega **at-least-once** en la práctica. Los reintentos van a pasar.

- Usá el id de CloudEvents (o una business key estable) como clave de dedupe.
- El efecto de negocio debe ser seguro ante repetición: Inbox, upsert, o "ya procesado → ACK".
- **Anti-pattern:** "Kafka me da exactly-once, no necesito idempotencia." Falso consuelo – EOS de Kafka no reemplaza el diseño del handler.

La clase 7 cubre la mecánica con Valkey; esta slide fija la regla para el TP.

Retries, DLQ y poison messages

- **Reintentos con backoff:** un fallo transitorio (timeout, servicio caído un segundo) se reintentará, espaciando los intentos.
- **Dead-letter queue (DLQ):** mensajes que fallaron N veces se mueven aparte – así no bloquean al resto.
- **Poison message:** payload corrupto o bug de deserialización. Reintentarlo para siempre no lo arregla – a la DLQ para revisión manual.

Anti-pattern: reintentar para siempre

En una cola clásica, un poison message + retry infinito bloquea a los que vienen detrás.

En **Kafka es peor de lo que parece**: un consumer que no avanza el offset en una partición deja **toda esa partición** atrasada (lag crece solo ahí). Los demás consumers del grupo siguen, pero esa key/entidad se pudre.

Regla: tope de intentos → DLQ (u otro topic) → alerta humana. Sin DLQ no hay resiliencia, hay denegación de servicio accidental.

Correlación: trazar la cadena async

Anti-pattern "async sin observabilidad" se arregla en el contrato del evento, no solo con un APM:

- **correlationId:** identifica el flujo de negocio entero (ej. orderId / id de Saga). Todos los eventos del mismo pedido lo comparten.
- **causationId:** id del evento (o request) que *causó* este. Arma el árbol padre→hijo.
- Viajan como extensiones CloudEvents; OpenTelemetry los retoma en la clase de Observabilidad.

Sin esto, debuggear la compensación de 3 saltos de la sección 4 es arqueología en 3 logs distintos.

Evolución de esquema en eventos

Los consumidores no se despliegan al mismo tiempo que el productor. El contrato tiene que tolerarlo.

- Versioná el type (`com.itba.order.created.v2`) o usá compatibilidad hacia atrás en el payload.
- **Additive changes** (campos nuevos opcionales) » renombrar/borrar campos.
- Un breaking change mal anunciado se convierte en **poison message** en cascada.

AsyncAPI + CloudEvents en el lab no son burocracia: son el contrato que evita ese silent break.

Lo que esta clase no repite

Ya lo vieron en la clase 7, con Valkey, en profundidad:

- **Semánticas de entrega:** at-most-once, at-least-once, exactly-once.
- **Idempotencia:** mecánica de claves y ventanas de falla (acá solo fijamos el contrato).
- **Ordenamiento bajo concurrencia** entre múltiples instancias.
- **Inbox** con Valkey – el patrón es el mismo que la slide de Inbox con DB.

Ese material sigue vivo en el deck estado-distribuido-patterns – vale la pena volver a repasarlo con la mecánica de Kafka fresca.

Cierre

Checklist de decisión

- ¿Necesitás la respuesta **ya**, para seguir? → síncrono.
- ¿Un evento, múltiples reacciones independientes? → pub/sub / EDA / fan-out (distintos consumer groups).
- ¿Necesitás replay o que varios consumidores lean el mismo historial? → Kafka.
- ¿Mensajería simple y liviana, sin gran volumen? → NATS.
- ¿Separar write path de lecturas optimizadas? → CQRS light sobre el mismo log.
- ¿Payload pesado o PII? → claim check, no fat event.
- ¿Una transacción de negocio cruza varios servicios? → Saga + **timeouts**.
 - Pocos pasos, equipos autónomos → **coreografía**.
 - Muchos pasos, necesitás visibilidad end-to-end → **orquestación** (Temporal, Camunda, Step Functions).
- ¿Dual write DB↔broker? → Outbox (+ CDC o poller) / Inbox en el consumidor.

Anti-patterns (1/2)

- **Monolito distribuido vía eventos:** todos escuchan todo, nadie entiende el flujo – coreografía sin límites.
- **God topic:** un solo topic events para todo el sistema → acoplamiento + consumidores ruidosos. Preferí topics/tipos acotados (type CloudEvents claro).
- **Chatty / ping-pong events:** A emite → B emite → A reacciona... RPC disfrazado de EDA.
- **Sync-over-async:** pedir respuesta inmediata sobre el canal async (o polling infernal / request-reply para todo).
- **Acoplamiento temporal disfrazado:** "es async" pero el flujo no avanza si el otro servicio no responde ya – faltaba un timeout o un modelo sync honesto.
- **Async sin observabilidad:** sin correlationId / causationId / traces, la cadena de 5 servicios es opaca.

Anti-patterns (2/2)

- **Dual write sin Outbox:** commit + publish "a mano" – el proceso se cae en el medio y el mundo queda inconsistente.
- **Asumir exactly-once:** saltarse Inbox/idempotencia porque "Kafka lo garantiza".
- **Key ausente o inestable:** orden roto entre eventos de la misma entidad; o partición caliente.
- **Retry forever / sin DLQ:** poison message frena la partición (o la cola) para todos.
- **Fat events / PII en el bus:** payloads enormes o datos sensibles viajando a N consumidores – usá claim check y minimizá el payload.
- **Ignorar consumer lag:** el broker "está verde" mientras el SLA de frescura ya explotó (o el disco se llena).
- **Orquestador-monolito:** un workflow que conoce 20 internos de cada servicio – acoplamiento central peor que la coreografía que reemplazó.
- **Happy-path async + compensación sync sin deadline:** liberar stock con un DELETE síncrono "cuando se pueda", sin timeout de Saga.

Recap

- Async cambia acoplamiento temporal por complejidad operativa – no es gratis; el **lag** es la factura.
- Queue vs pub/sub (en Kafka: mismo group vs distintos groups), CloudEvents y contratos que evolucionan son la base.
- CQRS light y claim check ordenan cómo se lee y qué viaja en el bus.
- La coreografía de clase 7 es EDA; hoy le pusimos broker, timeouts de Saga y el contraste con orquestación (Temporal).
- Kafka: key → partición, offsets, consumer groups – y los anti-patterns que nacen de usarlos mal.
- Outbox/CDC + Inbox + retries/DLQ + correlación cierran confiabilidad en el borde; idempotencia profunda quedó en Valkey (clase 7).

Para seguir profundizando

- Deck kafka: internals completos (segments, tuning de replicación, pipelines de datos, event sourcing).
- Deck estado-distribuido-patterns (clase 7): semánticas de entrega, idempotencia, Inbox con Valkey, ordenamiento.
- Temporal: orquestación de Sagas en producción (timers, compensación durable).

Fin!!